Python Interview Questions

List / Dict / Set Comprehensions — Top 100

Syntax · Filtering · Nesting · Performance · Dict & Set · Advanced Patterns

---

Section 1 — List Comprehensions: Fundamentals (Q1–Q25)

Q1. What is a list comprehension and what is its syntax?

A list comprehension is a concise way to build a list by expressing what each element should be, along with optional filtering, all in a single expression. Syntax: [expression for item in iterable if condition].

```
# Basic: square each number
squares = [x**2 for x in range(6)]
print(squares)   # [0, 1, 4, 9, 16, 25]

# With filter: only even squares
even_sq = [x**2 for x in range(6) if x % 2 == 0]
print(even_sq)   # [0, 4, 16]

# Equivalent for-loop
result = []
for x in range(6):
    if x % 2 == 0:
        result.append(x**2)
```

Q2. How does a list comprehension differ from a for loop?

A list comprehension is faster (the loop runs at C speed internally), more concise, and evaluates as a single expression that can be used inline. A for loop is more flexible for complex multi-step logic, side effects, or when intermediate state is needed.

```
import timeit

# List comprehension — runs loop in C
t1 = timeit.timeit(lambda: [x**2 for x in range(1000)], number=10000)

# for loop — pure Python
def with_loop():
    r = []
    for x in range(1000): r.append(x**2)
    return r
t2 = timeit.timeit(with_loop, number=10000)

print(f'Comprehension: {t1:.3f}s   Loop: {t2:.3f}s')
```

```
# Comprehension is typically ~30-50% faster
```

Q3. What is the scope of the loop variable in a list comprehension?

In Python 3, the loop variable in a list comprehension is local to the comprehension and does NOT leak into the enclosing scope. In Python 2 it did leak — this was fixed as a common bug source.

```
x = 'outer'
result = [x for x in range(3)]   # comprehension x is local
print(x)       # 'outer'  — unchanged in Python 3
print(result)  # [0, 1, 2]

# Contrast with a for loop (variable DOES leak)
for i in range(3): pass
print(i)   # 2  — loop variable leaks in all Python versions
```

Q4. How do you use a conditional expression (ternary) in a list comprehension?

Place an if/else expression in the output expression position (before for) to choose between two values. This is different from the trailing if which filters elements out.

```
# if/else in expression — transforms, keeps all elements
result = ['even' if x % 2 == 0 else 'odd' for x in range(6)]
print(result)   # ['even','odd','even','odd','even','odd']

# Trailing if — filters (removes odd numbers)
evens = [x for x in range(6) if x % 2 == 0]
print(evens)   # [0, 2, 4]

# Combining both
labelled_evens = [f'{x}:even' for x in range(6) if x % 2 == 0]
print(labelled_evens)   # ['0:even', '2:even', '4:even']
```

Q5. How do you write a nested list comprehension?

Add multiple for clauses to iterate over nested structures. They read left-to-right, just like nested for loops, with the leftmost for being the outermost loop.

```
# Flatten a 2D matrix
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [x for row in matrix for x in row]
print(flat)   # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Equivalent nested loops
flat2 = []
for row in matrix:
    for x in row:
        flat2.append(x)

# Cartesian product
```

```
pairs = [(x, y) for x in [1,2] for y in ['a','b']]
print(pairs)  # [(1,'a'),(1,'b'),(2,'a'),(2,'b')]
```

Q6. How do you create a 2D list (matrix) using a list comprehension?

Use a nested list comprehension: the outer loop generates rows and the inner loop generates columns within each row. This avoids the aliasing trap of [[0]*cols]*rows.

```
rows, cols = 3, 4

# Correct: each row is an independent list
matrix = [[0] * cols for _ in range(rows)]
matrix[0][1] = 99
print(matrix)   # [[0,99,0,0],[0,0,0,0],[0,0,0,0]]

# WRONG: all rows share the same list
bad = [[0] * cols] * rows
bad[0][1] = 99
print(bad)      # [[0,99,0,0],[0,99,0,0],[0,99,0,0]]
```

Q7. How do you use multiple if conditions in a list comprehension?

Chain multiple if clauses, or combine conditions with and/or inside a single if clause. Both are equivalent but chained ifs are sometimes clearer.

```
data = range(20)

# Chained ifs (and logic)
r1 = [x for x in data if x % 2 == 0 if x % 3 == 0]
print(r1)   # [0, 6, 12, 18]

# Equivalent single if with and
r2 = [x for x in data if x % 2 == 0 and x % 3 == 0]
print(r2)   # [0, 6, 12, 18]

# OR logic
r3 = [x for x in data if x < 3 or x > 16]
print(r3)   # [0, 1, 2, 17, 18, 19]
```

Q8. How do you apply a function to each element in a list comprehension?

Call the function in the expression position. The comprehension maps the function over all elements, equivalent to map() but often more readable.

```
words = ['hello', 'WORLD', 'Python']

# Apply method
lower  = [w.lower() for w in words]
print(lower)   # ['hello', 'world', 'python']
```

```
# Apply built-in
lengths = [len(w) for w in words]
print(lengths)  # [5, 5, 6]

# Apply custom function
import math
roots = [round(math.sqrt(x), 2) for x in range(1, 6)]
print(roots)    # [1.0, 1.41, 1.73, 2.0, 2.24]
```

Q9. How do you flatten a list of lists with a list comprehension?

Use a nested for clause: [item for sublist in outer for item in sublist]. This is O(n) and works for one level of nesting.

```
nested = [[1,2],[3,4],[5,6]]
flat = [x for lst in nested for x in lst]
print(flat)   # [1, 2, 3, 4, 5, 6]

# With filter: flatten and keep only positives
mixed = [[-1,2],[-3,4],[5,-6]]
pos = [x for sub in mixed for x in sub if x > 0]
print(pos)   # [2, 4, 5]

# Alternative: itertools.chain.from_iterable (faster for large lists)
import itertools
print(list(itertools.chain.from_iterable(nested)))
```

Q10. How do you use enumerate() inside a list comprehension?

Unpack the (index, value) pairs from enumerate() in the for clause. This gives access to both the position and the element.

```
words = ['apple', 'banana', 'cherry']

# Pair index with value
indexed = [(i, w) for i, w in enumerate(words)]
print(indexed)   # [(0,'apple'),(1,'banana'),(2,'cherry')]

# Keep only even-indexed elements
evens = [w for i, w in enumerate(words) if i % 2 == 0]
print(evens)   # ['apple', 'cherry']

# Build numbered list
numbered = [f'{i+1}. {w}' for i, w in enumerate(words)]
print(numbered)   # ['1. apple', '2. banana', '3. cherry']
```

Q11. How do you use zip() inside a list comprehension?

zip() pairs elements from multiple iterables; unpack the pairs in the for clause to get parallel iteration.

```
names  = ['Alice', 'Bob', 'Carol']
scores = [90, 85, 92]

# Pair name with score
report = [f'{n}: {s}' for n, s in zip(names, scores)]
print(report)   # ['Alice: 90', 'Bob: 85', 'Carol: 92']

# Element-wise sum of two lists
a = [1, 2, 3]
b = [10, 20, 30]
total = [x + y for x, y in zip(a, b)]
print(total)   # [11, 22, 33]
```

Q12. How do you remove duplicates from a list using a comprehension?

Use a set to track seen values inside the comprehension. Because comprehension variables are local, use a helper via assignment expression (:=) or a plain set + sorted approach.

```
# Simple: convert to set and back (loses order)
lst = [3, 1, 2, 1, 3, 4]
unique = list(set(lst))
print(sorted(unique))   # [1, 2, 3, 4]

# Order-preserving with walrus operator
seen = set()
unique_ordered = [x for x in lst if not (x in seen or seen.add(x))]
print(unique_ordered)   # [3, 1, 2, 4]

# dict.fromkeys preserves order (Python 3.7+)
unique2 = list(dict.fromkeys(lst))
print(unique2)   # [3, 1, 2, 4]
```

Q13. How do you transpose a matrix using a list comprehension?

Use nested comprehensions with zip(*matrix) to unzip (transpose) the rows into columns.

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

# Transpose using zip
transposed = [list(row) for row in zip(*matrix)]
print(transposed)
# [[1, 4, 7],
#  [2, 5, 8],
#  [3, 6, 9]]

# Manual nested comprehension
t2 = [[matrix[r][c] for r in range(3)] for c in range(3)]
print(t2 == transposed)   # True
```

Q14. What is a generator expression and how does it differ from a list comprehension?

A generator expression uses () instead of [] and is lazy — it produces values on demand without building the full list in memory. Use it when you only need to iterate once or when memory matters.

```
import sys

lc = [x**2 for x in range(10**6)]    # builds full list ~8MB
ge = (x**2 for x in range(10**6))    # lazy, ~200 bytes

print(sys.getsizeof(lc))   # ~8,448,728
print(sys.getsizeof(ge))   # 104

# Generator expression is consumed once
print(sum(ge))             # 333332833333500000
print(sum(ge))             # 0 — already exhausted
```

Q15. When should you prefer a list comprehension over map() or filter()?

Prefer list comprehensions when readability matters, when you need to apply multiple operations, or when using lambda with map/filter gets verbose. map/filter are slightly faster for simple built-in functions and return lazy iterators.

```
words = ['hello', 'world', 'python']

# map with built-in — slightly faster
upper_map = list(map(str.upper, words))

# Comprehension — clearer for transformations
upper_lc = [w.upper() for w in words]

print(upper_map == upper_lc)   # True

# filter equivalent
long_words = [w for w in words if len(w) > 5]
print(long_words)   # ['python']
```

Q16. How do you use list comprehensions with string operations?

String methods work as expressions inside comprehensions, making it easy to process lines of text, split/join, filter characters, or build formatted strings.

```
sentence = '  hello   world  python  '

# Split, strip, and capitalise
words = [w.capitalize() for w in sentence.split()]
print(words)   # ['Hello', 'World', 'Python']

# Filter vowels from a string
vowels = [c for c in 'hello world' if c in 'aeiou']
print(vowels)   # ['e', 'o', 'o']

# Build CSV row
row = ','.join([str(x) for x in [1, 2.5, 'three', True]])
```

```
print(row)   # '1,2.5,three,True'
```

Q17. How do you read and process a file using a list comprehension?

Use a list comprehension over an open file object (which is iterable line by line) to read, strip, and filter lines in one expression.

```
import io

# Simulate file content
content = io.StringIO('line1\n  line2  \n\nline3\n# comment\n')

# Read non-empty, non-comment lines, stripped
lines = [
    line.strip()
    for line in content
    if line.strip() and not line.startswith('#')
]
print(lines)   # ['line1', 'line2', 'line3']
```

Q18. How do list comprehensions handle exceptions?

List comprehensions do not have built-in exception handling. Use a helper function with try/except, a conditional expression to pre-validate, or write a regular loop for error-prone operations.

```
data = ['1', '2', 'three', '4', None]

# Helper function approach
def safe_int(x):
    try: return int(x)
    except (ValueError, TypeError): return None

result = [safe_int(x) for x in data]
print(result)   # [1, 2, None, 4, None]

# Filter out None after conversion
valid = [v for v in result if v is not None]
print(valid)    # [1, 2, 4]
```

Q19. What is the walrus operator (:=) and how does it help in comprehensions?

The walrus operator (PEP 572, Python 3.8+) assigns a value as part of an expression. In comprehensions, it lets you compute a value once and use it in both the filter condition and the output expression — avoiding double computation.

```
import math

data = range(-5, 6)
```

```
# Without walrus: sqrt computed twice (if filtered manually)
roots = [math.sqrt(x) for x in data if x >= 0]
print(roots)   # [0.0, 1.0, 1.41, 1.73, 2.0, 2.24]

# With walrus: compute once, use twice
results = [
    (x, y)
    for x in range(10)
    if (y := x**2 + 2*x - 3) > 0
]
print(results[:3])   # [(0, -3) filtered] [(2,5),(3,12),...]
```

Q20. How do you sort a list using a comprehension?

A list comprehension builds a list; sorting is done separately with sorted() or .sort(). Combine them in one expression using sorted() wrapping the comprehension.

```
data = [3, 1, 4, 1, 5, 9, 2, 6]

# Sort filtered result
sorted_evens = sorted([x for x in data if x % 2 == 0])
print(sorted_evens)   # [2, 4, 6]

# Sort with key inside comprehension result
words = ['banana', 'apple', 'cherry', 'date']
by_length = sorted([w for w in words if len(w) > 4], key=len)
print(by_length)   # ['apple', 'banana', 'cherry']
```

Q21. How do you use a list comprehension to unpack tuples?

Unpack tuples directly in the for clause by listing multiple targets, giving access to each element by name.

```
pairs = [(1,'a'), (2,'b'), (3,'c')]

# Unpack in for clause
sums   = [n + ord(c) for n, c in pairs]
print(sums)   # [98, 100, 102]

# Extract first elements
nums   = [n for n, _ in pairs]
print(nums)   # [1, 2, 3]

# Transform and repack
swapped = [(c, n) for n, c in pairs]
print(swapped)   # [('a',1),('b',2),('c',3)]
```

Q22. How do you use list comprehensions for data cleaning?

Chain transformations (strip, lower, replace) in the expression and use the filter clause to remove invalid entries — building a clean list in one pass.

```
raw = ['  Alice ', 'BOB', '', None, ' carol', '42', 'Dave  ']

cleaned = [
    name.strip().title()
    for name in raw
    if name and isinstance(name, str) and not name.strip().isdigit()
]
print(cleaned)   # ['Alice', 'Bob', 'Carol', 'Dave']
```

Q23. How do you build a list of indices using a comprehension?

Use enumerate() to access indices. Filter by condition to find the indices of elements that match.

```
lst = [10, 0, 30, 0, 50, 0]

# Find indices of all zeros
zero_indices = [i for i, v in enumerate(lst) if v == 0]
print(zero_indices)   # [1, 3, 5]

# Find indices of values above threshold
big = [i for i, v in enumerate(lst) if v > 20]
print(big)   # [2, 4]
```

Q24. How do you generate all primes up to N using a list comprehension?

Use a nested comprehension for the primality test. Readable but O(n*sqrt(n)) — for production use a sieve.

```
def is_prime(n):
    return n > 1 and all(n % i != 0 for i in range(2, int(n**0.5)+1))

N = 50
primes = [x for x in range(2, N+1) if is_prime(x)]
print(primes)
# [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47]

# Inline sieve-lite (less readable but no helper)
primes2 = [x for x in range(2, N+1)
           if all(x % d != 0 for d in range(2, int(x**0.5)+1))]
print(primes2 == primes)   # True
```

Q25. What are common pitfalls and anti-patterns in list comprehensions?

Key pitfalls: nesting too deep (>2 levels hurts readability), using comprehensions for side effects only, forgetting that the loop variable is local, and creating large intermediate lists when a generator expression would suffice.

```
# ANTI-PATTERN: comprehension purely for side effects
# [print(x) for x in range(5)]  # use a for loop instead

# ANTI-PATTERN: too deeply nested (unreadable)
# [x for a in b for x in a for y in x ...]  # use named loops

# ANTI-PATTERN: materialise large list just to sum
# sum([x**2 for x in range(10**7)])  # 80MB!
# Use generator: sum(x**2 for x in range(10**7))  # O(1) memory

# GOOD: readable, flat, purposeful
data = [1, 2, 3, 4, 5]
result = [x*2 for x in data if x % 2 != 0]
print(result)   # [2, 6, 10]
```

Section 2 — Dictionary Comprehensions (Q26–Q55)

Q26. What is a dictionary comprehension and what is its syntax?

A dict comprehension builds a dict by specifying key:value pairs in a single expression. Syntax: {key_expr: val_expr for item in iterable if condition}.

```
# Square each number as value, number as key
squares = {x: x**2 for x in range(6)}
print(squares)   # {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}

# Word -> length mapping
words = ['apple', 'banana', 'cherry']
lengths = {w: len(w) for w in words}
print(lengths)   # {'apple':5, 'banana':6, 'cherry':6}
```

Q27. How do you invert a dictionary using a comprehension?

Swap keys and values in the expression. Works cleanly only if values are unique and hashable. For non-unique values, use a set or list as the new value.

```
d = {'a': 1, 'b': 2, 'c': 3}

# Simple invert (unique values only)
inv = {v: k for k, v in d.items()}
print(inv)   # {1:'a', 2:'b', 3:'c'}

# Non-unique values: group keys by value
d2 = {'a': 1, 'b': 2, 'c': 1}
from collections import defaultdict
inv2 = defaultdict(list)
for k, v in d2.items(): inv2[v].append(k)
print(dict(inv2))   # {1:['a','c'], 2:['b']}
```

Q28. How do you filter a dictionary using a comprehension?

Iterate over .items() and add a filter condition. This builds a new filtered dict without modifying the original.

```
inventory = {'apple':5, 'banana':0, 'cherry':3, 'date':0}

# Keep only items with stock > 0
in_stock = {k: v for k, v in inventory.items() if v > 0}
print(in_stock)   # {'apple':5, 'cherry':3}

# Keep only string keys starting with a vowel
data = {'apple':1, 'banana':2, 'orange':3, 'grape':4}
vowel_keys = {k: v for k, v in data.items() if k[0] in 'aeiou'}
print(vowel_keys)   # {'apple':1, 'orange':3}
```

Q29. How do you transform dictionary values using a comprehension?

Iterate over .items() and apply a transformation to the value expression while keeping (or transforming) the key.

```
prices = {'apple': 1.50, 'banana': 0.75, 'cherry': 3.00}

# Apply 10% discount
discounted = {k: round(v * 0.9, 2) for k, v in prices.items()}
print(discounted)   # {'apple':1.35, 'banana':0.68, 'cherry':2.7}

# Normalise string values
raw = {'Name': ' Alice ', 'City': 'NEW YORK '}
clean = {k: v.strip().title() for k, v in raw.items()}
print(clean)   # {'Name':'Alice', 'City':'New York'}
```

Q30. How do you merge two dictionaries using a comprehension?

Iterate over both dicts together in a single comprehension. In Python 3.9+ you can use the | operator; for earlier Python, use ** unpacking or a comprehension.

```
d1 = {'a': 1, 'b': 2}
d2 = {'b': 20, 'c': 3}

# Comprehension merge (d2 wins on conflict)
merged = {k: v for d in (d1, d2) for k, v in d.items()}
print(merged)   # {'a':1, 'b':20, 'c':3}

# Python 3.9+ union operator
merged2 = d1 | d2
print(merged2)   # {'a':1, 'b':20, 'c':3}

# ** unpacking (Python 3.5+)
merged3 = {**d1, **d2}
print(merged3)   # {'a':1, 'b':20, 'c':3}
```

Q31. How do you build a frequency/histogram dict from a list?

Use collections.Counter for production, or build it manually with a comprehension over unique values and list.count().

```
from collections import Counter

data = ['a','b','a','c','b','a']

# Counter (fastest)
freq = Counter(data)
print(freq)   # Counter({'a':3, 'b':2, 'c':1})

# Comprehension approach
freq2 = {x: data.count(x) for x in set(data)}
print(freq2)   # {'a':3, 'c':1, 'b':2}

# Using defaultdict approach for clarity
from collections import defaultdict
freq3 = defaultdict(int)
for x in data: freq3[x] += 1
print(dict(freq3))   # {'a':3, 'b':2, 'c':1}
```

Q32. How do you group items by a property using a dict comprehension?

Group by iterating over unique keys and collecting items that match each key. For production, use itertools.groupby or collections.defaultdict.

```
people = [
    {'name':'Alice','dept':'Eng'},
    {'name':'Bob',  'dept':'HR' },
    {'name':'Carol','dept':'Eng'},
]

# Group names by dept
grouped = {
    dept: [p['name'] for p in people if p['dept'] == dept]
    for dept in {p['dept'] for p in people}
}
print(grouped)
# {'Eng':['Alice','Carol'], 'HR':['Bob']}
```

Q33. How do you create a dict from two lists using a comprehension?

Use zip() to pair the two lists element-wise inside the comprehension.

```
keys   = ['name', 'age', 'city']
values = ['Alice', 30, 'NYC']

# Dict comprehension with zip
```

```
d = {k: v for k, v in zip(keys, values)}
print(d)   # {'name':'Alice', 'age':30, 'city':'NYC'}

# Equivalent (often simpler)
d2 = dict(zip(keys, values))
print(d == d2)   # True

# dict() is usually preferred for this specific task
# Use comprehension when you need to transform key/value
d3 = {k.upper(): str(v) for k, v in zip(keys, values)}
print(d3)   # {'NAME':'Alice','AGE':'30','CITY':'NYC'}
```

Q34. How do you use nested dict comprehensions?

Write a dict comprehension as the value expression of an outer dict comprehension. Useful for building nested structures like config objects or lookup tables.

```
# Multiplication table as nested dict
table = {i: {j: i*j for j in range(1,4)} for i in range(1,4)}
print(table)
# {1:{1:1,2:2,3:3}, 2:{1:2,2:4,3:6}, 3:{1:3,2:6,3:9}}

# Access
print(table[2][3])   # 6

# Flatten nested dict
flat = {f'{ok}_{ik}': iv
        for ok, ov in table.items()
        for ik, iv in ov.items()}
print(list(flat.items())[:4])
# [('1_1',1),('1_2',2),('1_3',3),('2_1',2)]
```

Q35. How do you count characters in a string with a dict comprehension?

Use the string as the iterable and char as the key, counting occurrences with .count() or by iterating with a set of unique characters.

```
s = 'hello world'

# Comprehension with count() — O(n^2) but readable
freq = {c: s.count(c) for c in set(s)}
print(freq)   # {' ':1,'d':1,'e':1,'h':1,'l':3,'o':2,'r':1,'w':1}

# Counter — O(n) and more idiomatic
from collections import Counter
print(dict(Counter(s)))
```

Q36. How do you conditionally set values in a dict comprehension?

Use a ternary (if/else) in the value expression to choose between two values based on the current item.

```
scores = {'Alice':92, 'Bob':58, 'Carol':75, 'Dave':45}

# Grade based on score
grades = {
    name: 'Pass' if score >= 60 else 'Fail'
    for name, score in scores.items()
}
print(grades)
# {'Alice':'Pass','Bob':'Fail','Carol':'Pass','Dave':'Fail'}

# Three-way ternary
labels = {
    name: 'A' if s>=90 else ('B' if s>=70 else 'C')
    for name, s in scores.items()
}
print(labels)   # {'Alice':'A','Bob':'C','Carol':'B','Dave':'C'}
```

Q37. How do you swap keys and values only for items meeting a condition?

Filter in the condition clause and swap in the expression, building a new dict with only the qualifying items inverted.

```
d = {'a':1, 'b':2, 'c':3, 'd':4}

# Invert only where value is even
inv_even = {v: k for k, v in d.items() if v % 2 == 0}
print(inv_even)   # {2:'b', 4:'d'}

# Keep originals for odd, invert evens — combine two comprehensions
mixed = {
    **{k: v for k, v in d.items() if v % 2 != 0},
    **{v: k for k, v in d.items() if v % 2 == 0},
}
print(mixed)   # {'a':1,'c':3, 2:'b', 4:'d'}
```

Q38. How do you normalise dictionary keys (e.g., lowercase all keys)?

Iterate over .items() and transform the key expression, keeping the value unchanged.

```
raw = {'Name':'Alice', 'AGE':30, 'City':'NYC', 'ZIP':10001}

# Lowercase keys
normalised = {k.lower(): v for k, v in raw.items()}
print(normalised)
# {'name':'Alice','age':30,'city':'NYC','zip':10001}

# Snake_case keys (replace space/hyphen with underscore)
messy = {'First Name':'Alice', 'Last-Name':'Smith'}
snake = {k.lower().replace(' ','_').replace('-','_'): v
         for k, v in messy.items()}
print(snake)   # {'first_name':'Alice','last_name':'Smith'}
```

Q39. How do you build a lookup table from a list using a dict comprehension?

Use the list elements as keys and compute derived values (e.g., index, transformation) as values for O(1) lookup later.

```
words = ['apple', 'banana', 'cherry', 'date']

# word -> index lookup
word_to_idx = {w: i for i, w in enumerate(words)}
print(word_to_idx)   # {'apple':0,'banana':1,'cherry':2,'date':3}
print(word_to_idx['cherry'])   # 2  — O(1) lookup

# word -> length lookup
word_to_len = {w: len(w) for w in words}

# Sort words by their lookup value (length)
by_length = sorted(words, key=lambda w: word_to_len[w])
print(by_length)   # ['date','apple','banana','cherry']
```

Q40. How do you handle duplicate keys in a dict comprehension?

If multiple iterations produce the same key, the last value wins silently. To detect or handle duplicates, use a pre-check or accumulate values in a list.

```
# Last value wins
data = [('a',1),('b',2),('a',3)]
d = {k: v for k, v in data}
print(d)   # {'a':3, 'b':2}   — first 'a':1 overwritten

# Detect and accumulate duplicates
from collections import defaultdict
acc = defaultdict(list)
for k, v in data: acc[k].append(v)
print(dict(acc))   # {'a':[1,3], 'b':[2]}
```

Q41. How do you flatten a nested dict using a comprehension?

Recursion or a helper to yield key-path/value pairs, then collect into a flat dict with dotted keys.

```
def flatten_dict(d, prefix=''):
    items = {}
    for k, v in d.items():
        new_key = f'{prefix}.{k}' if prefix else k
        if isinstance(v, dict):
            items.update(flatten_dict(v, new_key))
        else:
            items[new_key] = v
    return items

nested = {'a': 1, 'b': {'c': 2, 'd': {'e': 3}}}
print(flatten_dict(nested))
# {'a':1, 'b.c':2, 'b.d.e':3}
```

Q42. How do you use dict comprehensions with enumerate()?

enumerate() gives (index, value) pairs; use them to map each value to its index or vice versa.

```
fruits = ['apple', 'banana', 'cherry']

# Index -> fruit
idx_to_fruit = {i: f for i, f in enumerate(fruits)}
print(idx_to_fruit)   # {0:'apple', 1:'banana', 2:'cherry'}

# Fruit -> index (for O(1) lookup)
fruit_to_idx = {f: i for i, f in enumerate(fruits)}
print(fruit_to_idx['cherry'])   # 2
```

Q43. How do you compute a running total or cumulative mapping with a dict comprehension?

Use itertools.accumulate to compute running values, then zip original keys with accumulated values.

```
import itertools

monthly = {'Jan':100, 'Feb':150, 'Mar':200, 'Apr':120}
keys = list(monthly.keys())
vals = list(monthly.values())

cumulative = {k: v for k, v in zip(keys, itertools.accumulate(vals))}
print(cumulative)
# {'Jan':100, 'Feb':250, 'Mar':450, 'Apr':570}
```

Q44. What is the difference between dict() and a dict comprehension?

dict(zip(k, v)) and dict([(k,v)]) work for direct key-value pairing. A dict comprehension is more powerful: it allows arbitrary key and value expressions, filtering, and is generally faster than constructing a list of tuples first.

```
keys = ['a','b','c']
vals = [1, 2, 3]

# dict with zip — clean for simple mapping
d1 = dict(zip(keys, vals))

# Comprehension — adds transformation power
d2 = {k: v*10 for k, v in zip(keys, vals)}

print(d1)   # {'a':1,'b':2,'c':3}
print(d2)   # {'a':10,'b':20,'c':30}
```

Q45. How do you build a dict comprehension from a CSV string?

Split the CSV string into rows and fields, then unpack field pairs into key:value in the comprehension.

```
csv_data = 'name:Alice,age:30,city:NYC'

# Parse key:value pairs from comma-separated string
parsed = {pair.split(':')[0]: pair.split(':')[1]
          for pair in csv_data.split(',')}
print(parsed)   # {'name':'Alice','age':'30','city':'NYC'}

# Typed conversion
def try_int(v):
    try: return int(v)
    except ValueError: return v

typed = {k: try_int(v) for k, v in parsed.items()}
print(typed)   # {'name':'Alice','age':30,'city':'NYC'}
```

Q46. How do you use dict comprehensions to pivot data?

Transform a list of records into a column-oriented dict, where each key is a column name and each value is a list of that column's values.

```
rows = [
    {'name':'Alice','score':90,'grade':'A'},
    {'name':'Bob',  'score':75,'grade':'B'},
    {'name':'Carol','score':85,'grade':'A'},
]

# Pivot: column name -> list of values
cols = rows[0].keys()
pivoted = {col: [row[col] for row in rows] for col in cols}
print(pivoted)
# {'name':['Alice','Bob','Carol'],
#  'score':[90,75,85],
#  'grade':['A','B','A']}
```

Q47. How do you merge dicts with a conflict resolution strategy?

Use a dict comprehension iterating over all key-value pairs from both dicts, applying a function to resolve conflicts when the same key appears in both.

```
d1 = {'a':10, 'b':20, 'c':30}
d2 = {'b':5,  'c':40, 'd':50}

# Resolve by summing conflicting values
merged = {
    k: d1.get(k, 0) + d2.get(k, 0)
    for k in d1.keys() | d2.keys()
}
print(merged)   # {'a':10,'b':25,'c':70,'d':50}

# Resolve by taking the max
merged_max = {
    k: max(d1.get(k,0), d2.get(k,0))
    for k in d1.keys() | d2.keys()
}
print(merged_max)   # {'a':10,'b':20,'c':40,'d':50}
```

Q48. How do you select specific keys from a dictionary?

Use a comprehension iterating over the desired keys, fetching each value with dict[key]. Use .get() to handle missing keys gracefully.

```
user = {'id':1,'name':'Alice','email':'a@b.com','password':'secret','age':30}

# Select only public fields
public_fields = ['id', 'name', 'email', 'age']
public = {k: user[k] for k in public_fields if k in user}
print(public)
# {'id':1,'name':'Alice','email':'a@b.com','age':30}
```

Q49. How do you rename keys in a dictionary using a comprehension?

Provide a mapping from old key to new key, then use it in the comprehension's key expression.

```
record = {'fname':'Alice', 'lname':'Smith', 'dob':'1990-01-01'}

rename_map = {'fname':'first_name', 'lname':'last_name', 'dob':'birth_date'}

renamed = {rename_map.get(k, k): v for k, v in record.items()}
print(renamed)
# {'first_name':'Alice','last_name':'Smith','birth_date':'1990-01-01'}
```

Q50. How do you diff two dicts using a comprehension?

Build three dicts: keys only in dict1, keys only in dict2, and keys in both with different values.

```
old = {'a':1, 'b':2, 'c':3, 'd':4}
new = {'b':2, 'c':99, 'd':4, 'e':5}

added    = {k: new[k] for k in new if k not in old}
removed  = {k: old[k] for k in old if k not in new}
changed  = {k: (old[k], new[k]) for k in old
            if k in new and old[k] != new[k]}

print('Added:',   added)    # {'e':5}
print('Removed:', removed)  # {'a':1}
print('Changed:', changed)  # {'c':(3,99)}
```

Q51. How do you create a sparse matrix as a dict comprehension?

Represent a matrix as a dict with (row, col) tuple keys, storing only non-zero values. Comprehension over the enumerated matrix rows and columns.

```
matrix = [[0,1,0],[2,0,3],[0,0,4]]

sparse = {
    (r, c): val
    for r, row in enumerate(matrix)
    for c, val in enumerate(row)
    if val != 0
}
print(sparse)
# {(0,1):1,(1,0):2,(1,2):3,(2,2):4}

# O(1) lookup
print(sparse.get((1,2), 0))   # 3
print(sparse.get((0,0), 0))   # 0  (not stored)
```

Q52. How do you build a reverse index using a dict comprehension?

A reverse index maps each value (e.g., word) to a list of positions (e.g., document ids). Use defaultdict or accumulate in a comprehension over unique values.

```
docs = {
    0: ['python','fast','easy'],
    1: ['java','fast','verbose'],
    2: ['python','popular'],
}

# word -> [doc_ids]
all_words = {w for words in docs.values() for w in words}
index = {
    word: [doc_id for doc_id, words in docs.items() if word in words]
    for word in all_words
}
print(index['python'])   # [0, 2]
print(index['fast'])     # [0, 1]
```

Q53. How do you use dict comprehensions with defaultdict?

You cannot directly create a defaultdict with a comprehension, but you can convert a comprehension result to defaultdict, or build a regular dict comprehension and then wrap it.

```
from collections import defaultdict

data = ['cat','car','card','care','bat','bar']

# Group by first letter using defaultdict
by_letter = defaultdict(list)
for w in data: by_letter[w[0]].append(w)
print(dict(by_letter))
# {'c':['cat','car','card','care'],'b':['bat','bar']}

# Equivalent with comprehension (less efficient for grouping)
by_letter2 = {
    letter: [w for w in data if w.startswith(letter)]
    for letter in {w[0] for w in data}
}
```

```
print(by_letter2)
```

Q54. How do you compute set operations on dict keys using comprehensions?

Dict keys support set operations (|, &, -, ^). Use these in comprehensions to build intersection, union, difference, or symmetric difference dicts.

```
a = {'x':1,'y':2,'z':3}
b = {'y':20,'z':30,'w':40}

# Intersection: keys in both (take value from a)
inter = {k: a[k] for k in a.keys() & b.keys()}
print(inter)   # {'y':2, 'z':3}

# Difference: keys only in a
diff = {k: a[k] for k in a.keys() - b.keys()}
print(diff)    # {'x':1}

# Symmetric difference: keys in exactly one
sym = {k: a.get(k, b[k]) for k in a.keys() ^ b.keys()}
print(sym)     # {'x':1,'w':40}
```

Q55. What are common pitfalls with dict comprehensions?

Key pitfalls: silent key collision (last value wins), unhashable key types, mutating the source dict during iteration, and using mutable objects as values without copying.

```
# Pitfall 1: silent collision
pairs = [('a',1),('b',2),('a',99)]
d = {k: v for k, v in pairs}
print(d)   # {'a':99,'b':2} — first 'a' silently lost

# Pitfall 2: unhashable key
# d = {[1,2]: 'val'}   # TypeError: unhashable type: 'list'
# Fix: convert to tuple
d = {tuple([1,2]): 'val'}   # OK

# Pitfall 3: modifying dict during iteration
# {k: v for k, v in d.items() if d.pop(k)}  # RuntimeError
# Fix: iterate over list(d.items())
```

Section 3 — Set Comprehensions (Q56–Q70)

Q56. What is a set comprehension and what is its syntax?

A set comprehension builds a set by expressing elements in a single expression, automatically deduplicating. Syntax: {expression for item in iterable if condition}.

```
# Unique squares
sq = {x**2 for x in [-3,-2,-1,0,1,2,3]}
print(sq)   # {0, 1, 4, 9}  — duplicates removed

# Unique first characters
words = ['apple','avocado','banana','berry','cherry']
first_chars = {w[0] for w in words}
print(first_chars)   # {'a','b','c'} — order not guaranteed
```

Q57. How does a set comprehension differ from a list comprehension?

Set comprehensions use {} and produce an unordered, deduplicated collection. List comprehensions use [] and preserve order and duplicates. Use set comprehensions when uniqueness matters and order does not.

```
data = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

lc = [x for x in data]   # [3,1,4,1,5,9,2,6,5,3] — preserves all
sc = {x for x in data}   # {1,2,3,4,5,6,9}         — unique only

print(len(lc))   # 10
print(len(sc))   # 7

# Check membership O(1) with set
print(5 in sc)   # True
```

Q58. How do you find unique values in a list using a set comprehension?

Pass the list directly into a set comprehension — every element is unique in the result. Equivalent to set(lst) but allows transformation and filtering.

```
emails = ['a@x.com','B@X.COM','a@x.com','c@y.com','b@x.com']

# Unique normalised emails
unique = {e.lower() for e in emails}
print(unique)   # {'a@x.com','b@x.com','c@y.com'}

# Unique domains
domains = {e.split('@')[1].lower() for e in emails}
print(domains)  # {'x.com','y.com'}
```

Q59. How do you use set operations on set comprehension results?

The result is a regular Python set supporting union (|), intersection (&), difference (-), and symmetric difference (^).

```
evens = {x for x in range(20) if x % 2 == 0}
```

```
threes = {x for x in range(20) if x % 3 == 0}

print(evens & threes)    # {0,6,12,18}   — divisible by 6
print(evens | threes)    # union
print(evens - threes)    # {2,4,8,10,14,16}   — even, not mult of 3
print(evens ^ threes)    # symmetric difference
```

Q60. How do you find common elements between two lists using a set comprehension?

Convert one list to a set for O(1) membership testing, then use a set comprehension over the other list.

```
list1 = [1, 2, 3, 4, 5, 6]
list2 = [4, 5, 6, 7, 8, 9]

common = {x for x in list1 if x in set(list2)}
print(common)   # {4, 5, 6}

# Equivalent using set intersection
print(set(list1) & set(list2))   # {4, 5, 6}
```

Q61. How do you use a set comprehension to extract unique words from text?

Split the text into words, normalise (lowercase, strip punctuation), and collect into a set comprehension for unique terms.

```
import re

text = 'The quick brown fox jumps over the lazy dog. The dog barked.'

# Unique normalised words
words = {w.lower() for w in re.findall(r'[a-zA-Z]+', text)}
print(sorted(words))
# ['barked','brown','dog','fox','jumps','lazy','over','quick','the']

print(len(words))   # 9 unique words
```

Q62. How do you use nested set comprehensions?

The expression inside a set comprehension can itself be any hashable expression. Note that sets cannot contain mutable elements (lists, dicts, sets) — use tuples if needed.

```
# All unique pairwise sums from two lists
a = [1, 2, 3]
b = [10, 20, 30]

sums = {x + y for x in a for y in b}
print(sums)   # {11,12,13,21,22,23,31,32,33}

# Unique frozensets (pairs, regardless of order)
lst = [1, 2, 3, 4]
```

```
pairs = {frozenset({x, y}) for x in lst for y in lst if x != y}
print(len(pairs))   # 6 unique unordered pairs
```

Q63. How do you filter a set comprehension using multiple conditions?

Chain multiple if clauses or use boolean operators in a single if. Both behave identically to list comprehension filtering.

```
data = range(50)

# Elements divisible by 2 AND 3 (multiples of 6)
s1 = {x for x in data if x % 2 == 0 if x % 3 == 0}
print(s1)   # {0,6,12,18,24,30,36,42,48}

# Elements where digit sum > 5
s2 = {x for x in data if sum(int(d) for d in str(x)) > 5}
print(sorted(s2)[:5])   # [6, 7, 8, 9, 16]
```

Q64. Why can't you have a set of lists? What do you use instead?

Lists are mutable and therefore unhashable — they cannot be members of a set. Use tuples (immutable) or frozensets as hashable alternatives.

```
# ERROR
# s = {[1,2], [3,4]}   # TypeError: unhashable type: 'list'

# Fix 1: tuples
s = {(1,2), (3,4), (1,2)}
print(s)   # {(1,2),(3,4)}

# Fix 2: frozensets (when order doesn't matter)
s2 = {frozenset({1,2}), frozenset({3,4}), frozenset({2,1})}
print(len(s2))   # 2  — frozenset({1,2})==frozenset({2,1})
```

Q65. How do you convert a set comprehension result to a sorted list?

Wrap the set comprehension in sorted(). Sets are unordered; sorted() always returns a list.

```
data = [5, 3, 1, 4, 2, 3, 5, 1]

unique_sorted = sorted({x for x in data})
print(unique_sorted)   # [1, 2, 3, 4, 5]

# With key
words = ['banana','Apple','cherry','apple']
unique_words_sorted = sorted({w.lower() for w in words})
print(unique_words_sorted)   # ['apple','banana','cherry']
```

Q66. How do you check if a set comprehension result is a subset or superset?

Use .issubset(), .issuperset(), or operators <= and >= to compare the result set against another set.

```
evens = {x for x in range(20) if x % 2 == 0}
small = {0, 2, 4}

print(small.issubset(evens))     # True
print(small <= evens)            # True
print(evens.issuperset(small))   # True
print(evens >= small)            # True

print({x for x in range(5)} <= evens)   # False — includes odds
```

Q67. How do you use a set comprehension to find elements appearing in all sub-lists?

Use a reduce with intersection, or initialise with the first set and intersect with the rest in a comprehension.

```
from functools import reduce

lists = [[1,2,3,4],[2,3,5],[2,3,6,7],[3,2,8]]

# reduce with intersection
common = reduce(lambda a, b: a & b, (set(lst) for lst in lists))
print(common)   # {2, 3}

# Pythonic one-liner
common2 = set.intersection(*map(set, lists))
print(common2)   # {2, 3}
```

Q68. What is the performance advantage of set comprehensions over list comprehensions for membership testing?

Membership in a set is O(1) average due to hashing; in a list it is O(n). Build a set once and reuse it for multiple lookups — much faster than scanning a list each time.

```
import timeit

N = 10_000
big_list = list(range(N))
big_set  = {x for x in big_list}

# O(n) list lookup
t1 = timeit.timeit(lambda: 9999 in big_list, number=100000)

# O(1) set lookup
t2 = timeit.timeit(lambda: 9999 in big_set,  number=100000)

print(f'List: {t1:.4f}s   Set: {t2:.4f}s')
# Set is typically 100-500x faster for large N
```

Q69. How do you use a set comprehension to validate data?

Build a set of valid values and check the dataset against it, or build a set of invalid entries to report.

```
VALID_STATUSES = {'active', 'inactive', 'pending', 'suspended'}

records = [
    {'id':1,'status':'active'},
    {'id':2,'status':'unknown'},
    {'id':3,'status':'inactive'},
    {'id':4,'status':'deleted'},
]

# Find unique invalid statuses
invalid = {r['status'] for r in records
           if r['status'] not in VALID_STATUSES}
print(invalid)   # {'unknown','deleted'}

# IDs with invalid status
bad_ids = {r['id'] for r in records
           if r['status'] not in VALID_STATUSES}
print(bad_ids)   # {2, 4}
```

Q70. How do you combine list, dict, and set comprehensions?

You can nest or combine comprehensions freely. A common pattern: use a set comprehension to deduplicate, list comprehension to order, and dict comprehension to look up.

```
data = [
    {'name':'Alice','tags':['python','data']},
    {'name':'Bob',  'tags':['java','data','cloud']},
    {'name':'Carol','tags':['python','cloud']},
]

# All unique tags across all users (set comprehension)
all_tags = {tag for user in data for tag in user['tags']}
print(sorted(all_tags))   # ['cloud','data','java','python']

# tag -> [users who have it] (dict of list comprehensions)
tag_map = {
    tag: [u['name'] for u in data if tag in u['tags']]
    for tag in all_tags
}
print(tag_map['python'])   # ['Alice','Carol']
```

Section 4 — Advanced Patterns & Performance (Q71–Q100)

Q71. How do you use comprehensions for matrix multiplication?

Implement the dot product with a nested comprehension over rows and columns, using sum() with a generator expression for each cell.

```
def matmul(A, B):
    rows_A, cols_A = len(A), len(A[0])
    cols_B = len(B[0])
    return [
        [sum(A[i][k] * B[k][j] for k in range(cols_A))
         for j in range(cols_B)]
        for i in range(rows_A)
    ]

A = [[1,2],[3,4]]
B = [[5,6],[7,8]]
print(matmul(A, B))   # [[19,22],[43,50]]
```

Q72. How do you implement a Caesar cipher using a list comprehension?

Shift each letter by a fixed amount within a-z or A-Z using ord/chr arithmetic in a comprehension.

```
def caesar(text, shift):
    def shift_char(c):
        if c.isalpha():
            base = ord('A') if c.isupper() else ord('a')
            return chr((ord(c) - base + shift) % 26 + base)
        return c
    return ''.join([shift_char(c) for c in text])

print(caesar('Hello, World!', 3))   # 'Khoor, Zruog!'
print(caesar('Khoor, Zruog!', -3))  # 'Hello, World!'
```

Q73. How do you use comprehensions for string parsing and tokenisation?

Split, strip, and classify tokens in one comprehension pass over the input string.

```
expr = ' 3 + 42 * ( 7 - 1 ) '

# Tokenise: split and classify each token
tokens = [
    ('NUM', int(t)) if t.isdigit() else ('OP', t)
    for t in expr.split()
    if t.strip()
]
print(tokens)
# [('NUM',3),('OP','+'),('NUM',42),('OP','*'),
#  ('OP','('),('NUM',7),('OP','-'),('NUM',1),('OP',')')]
```

Q74. How do you use comprehensions in data science pipelines?

Comprehensions handle feature extraction, normalisation, and one-hot encoding concisely — great for preprocessing before feeding into NumPy or pandas.

```
data = [
    {'age':25,'city':'NYC','income':50000},
```

```
    {'age':35,'city':'LA', 'income':80000},
    {'age':28,'city':'NYC','income':60000},
]

# Normalise income (min-max)
incomes = [d['income'] for d in data]
mn, mx = min(incomes), max(incomes)
norm = [(d['income']-mn)/(mx-mn) for d in data]
print([round(x,2) for x in norm])   # [0.0, 1.0, 0.33]

# One-hot encode city
cities = {d['city'] for d in data}
one_hot = [{c: int(d['city']==c) for c in cities} for d in data]
print(one_hot[0])   # {'NYC':1,'LA':0}
```

Q75. How do you benchmark comprehensions vs loops?

Use timeit to compare approaches for the same task. Comprehensions consistently outperform equivalent for-loops for simple transformations due to reduced bytecode overhead.

```
import timeit

N = 100_000

def loop():
    r = []
    for x in range(N):
        if x % 2 == 0:
            r.append(x**2)
    return r

lc_time   = timeit.timeit(lambda: [x**2 for x in range(N) if x%2==0], number=100)
loop_time = timeit.timeit(loop, number=100)

print(f'LC: {lc_time:.3f}s   Loop: {loop_time:.3f}s')
# LC is typically 20-40% faster
```

Q76. How do you generate a multiplication table using comprehensions?

A nested list comprehension generates rows, and f-strings or formatting format each cell.

```
N = 5

# As nested list
table = [[i*j for j in range(1, N+1)] for i in range(1, N+1)]
for row in table:
    print(row)
# [1,2,3,4,5]
# [2,4,6,8,10]  ...

# Formatted string table
formatted = ['\t'.join(str(i*j) for j in range(1, N+1))
             for i in range(1, N+1)]
print('\n'.join(formatted))
```

Q77. How do you use comprehensions to implement a simple pipeline pattern?

Compose a series of comprehensions where each feeds into the next — similar to a generator pipeline but eager (building lists at each stage).

```
raw = ['  10', ' -5', '20 ', 'abc', '  7  ', '-999']

# Stage 1: strip whitespace
stripped = [s.strip() for s in raw]

# Stage 2: filter numeric
numeric  = [s for s in stripped if s.lstrip('-').isdigit()]

# Stage 3: convert to int
integers = [int(s) for s in numeric]

# Stage 4: keep positives
positives = [x for x in integers if x > 0]

print(positives)   # [10, 20, 7]
```

Q78. How do you use comprehensions to work with JSON-like nested structures?

Comprehensions extract, transform, or validate nested JSON-like data — iterating over lists of dicts and applying nested key access.

```
users = [
    {'id':1,'profile':{'name':'Alice','active':True},'scores':[90,85]},
    {'id':2,'profile':{'name':'Bob',  'active':False},'scores':[70,75]},
    {'id':3,'profile':{'name':'Carol','active':True},'scores':[95,98]},
]

# Names of active users with avg score > 80
result = [
    u['profile']['name']
    for u in users
    if u['profile']['active']
    and sum(u['scores'])/len(u['scores']) > 80
]
print(result)   # ['Alice', 'Carol']
```

Q79. How do you implement set difference and symmetric difference with comprehensions?

While set operators are cleaner, you can express difference and symmetric difference with comprehensions for custom filtering logic.

```
a = {1,2,3,4,5}
b = {3,4,5,6,7}
```

```
# Difference a - b
diff = {x for x in a if x not in b}
print(diff)   # {1, 2}

# Symmetric difference
sym = {x for x in a | b if x not in a & b}
print(sym)    # {1, 2, 6, 7}

# Using operators (simpler)
print(a - b)   # {1, 2}
print(a ^ b)   # {1, 2, 6, 7}
```

Q80. How do you combine comprehensions with functools.reduce?

Use a comprehension to prepare the collection, then reduce to fold it into a single value. Alternatively, nest the comprehension inside the reduce call.

```
from functools import reduce

matrix = [[1,2,3],[4,5,6],[7,8,9]]

# Flatten then reduce to product of all elements
flat  = [x for row in matrix for x in row]
product = reduce(lambda a, b: a * b, flat)
print(product)   # 362880 = 9!

# Row-sums then overall max
row_sums = [sum(row) for row in matrix]
print(max(row_sums))   # 24
```

Q81. How do list comprehensions behave inside functions vs at module level?

In both cases, comprehension variables are local to the comprehension scope. However, comprehensions at module level execute at import time; inside functions, they execute at call time. Large module-level comprehensions slow imports.

```
# Module level: runs at import — avoid large computations
PRIMES = [x for x in range(2, 100)
          if all(x%d!=0 for d in range(2,int(x**0.5)+1))]

# Function level: runs on each call
def get_squares(n):
    return [x**2 for x in range(n)]

# Lazy alternative: use a function/generator at module level
def primes_up_to(n):
    return [x for x in range(2, n)
            if all(x%d!=0 for d in range(2, int(x**0.5)+1))]
```

Q82. How do you use comprehensions for bit manipulation?

Use bitwise operators in the expression and filter clauses to extract, transform, or select based on bit patterns.

```
n = 0b10110101   # 181

# List all set bits (positions where bit=1)
set_bits = [i for i in range(8) if (n >> i) & 1]
print(set_bits)   # [0, 2, 4, 5, 7]

# Mask lower 4 bits of each number
nums = [0xFF, 0xAB, 0x12, 0x7C]
lower_nibbles = [x & 0x0F for x in nums]
print(lower_nibbles)   # [15, 11, 2, 12]
```

Q83. What is the difference between a comprehension and a map+filter chain?

A comprehension is a single syntactic construct; map+filter is a chain of two lazy iterators. Comprehensions are more readable for complex logic. map+filter avoids function call overhead for simple built-in operations.

```
data = range(20)

# Comprehension
result_lc = [x**2 for x in data if x % 3 == 0]

# map + filter
result_mf = list(map(lambda x: x**2, filter(lambda x: x % 3 == 0, data)))

print(result_lc == result_mf)   # True

# For built-ins, map+filter can be faster:
# list(map(abs, filter(None, data)))
```

Q84. How do you use comprehensions for anagram detection?

Use a dict comprehension (or Counter) to build a character frequency map and compare across words.

```
from collections import Counter

words = ['listen','silent','enlist','inlets','hello','world']

# Group anagrams using sorted string as key
key = lambda w: ''.join(sorted(w))
unique_keys = {key(w) for w in words}
groups = {k: [w for w in words if key(w)==k] for k in unique_keys}
print([g for g in groups.values() if len(g) > 1])
# [['listen','silent','enlist','inlets']]
```

Q85. How do you implement a simple tokenizer with a dict comprehension?

Map each token to an integer ID — common in NLP preprocessing.

```
corpus = ['the cat sat on the mat the cat']

# Build vocabulary (unique words)
vocab = sorted({word for text in corpus for word in text.split()})

# Word -> index
word2idx = {word: idx for idx, word in enumerate(vocab)}
print(word2idx)   # {'cat':0,'mat':1,'on':2,'sat':3,'the':4}

# Index -> word
idx2word = {idx: word for word, idx in word2idx.items()}

# Tokenise
tokens = [word2idx[w] for w in corpus[0].split()]
print(tokens)   # [4,0,3,2,4,1,4,0]
```

Q86. How do you use dict comprehensions for environment variable parsing?

Parse OS environment variables into a typed dict, filtering and transforming as needed.

```
import os

# Simulate environment
env = {'APP_PORT':'8080','APP_DEBUG':'true','APP_NAME':'myapp',
       'PATH':'/usr/bin','HOME':'/home/user'}

# Extract APP_* variables, strip prefix, type-cast
def cast(v):
    if v.isdigit(): return int(v)
    if v.lower() in ('true','false'): return v.lower()=='true'
    return v

config = {
    k[4:].lower(): cast(v)
    for k, v in env.items()
    if k.startswith('APP_')
}
print(config)   # {'port':8080,'debug':True,'name':'myapp'}
```

Q87. How do you use comprehensions with regular expressions?

Apply re.match, re.search, or re.findall inside comprehensions to filter or extract matched content from a collection of strings.

```
import re

emails = ['alice@example.com','not-an-email','bob@test.org',
          'invalid@','carol@domain.co.uk']

pattern = re.compile(r'^[\w.+-]+@[\w-]+\.[a-z]{2,}$')
```

```
# Filter valid emails
valid = [e for e in emails if pattern.match(e)]
print(valid)   # ['alice@example.com','bob@test.org','carol@domain.co.uk']

# Extract domains
domains = {m.group(1) for e in valid if (m := re.search(r'@(.+)$', e))}
print(domains)   # {'example.com','test.org','domain.co.uk'}
```

Q88. How do you implement a word frequency counter with a dict comprehension?

Build a frequency dict using Counter for efficiency, or with a comprehension over unique words and .count().

```
text = 'to be or not to be that is the question to be'
words = text.split()

# Comprehension approach
freq = {w: words.count(w) for w in set(words)}
print(freq)   # {'to':3,'be':3,'or':1,...}

# Most common words
top3 = sorted(freq.items(), key=lambda kv: -kv[1])[:3]
print(top3)   # [('to',3),('be',3),('or',1)] (or similar tie-breaking)
```

Q89. How do comprehensions interact with memory in CPython?

Each list/dict/set comprehension creates its own code object and local scope. Generator expressions are cheaper: they create a generator object (~200 bytes) regardless of size. Use sys.getsizeof to verify.

```
import sys

n = 100_000

lc = [x for x in range(n)]
ge = (x for x in range(n))
sc = {x for x in range(n)}
dc = {x: x for x in range(n)}

print(sys.getsizeof(lc))   # ~824456 bytes
print(sys.getsizeof(ge))   # ~104 bytes
print(sys.getsizeof(sc))   # ~4194384 bytes
print(sys.getsizeof(dc))   # ~4194392 bytes
```

Q90. How do you use comprehensions for graph adjacency list construction?

Build a dict mapping each node to its neighbours from a list of edges using a dict of set comprehensions.

```
edges = [(1,2),(1,3),(2,3),(3,4),(4,5)]
```

```
nodes = {n for e in edges for n in e}

# Undirected adjacency list
adj = {
    n: {b for a,b in edges if a==n} | {a for a,b in edges if b==n}
    for n in nodes
}
print(adj)
# {1:{2,3},2:{1,3},3:{1,2,4},4:{3,5},5:{4}}

# BFS reachable from node 1
from collections import deque
visited = set()
q = deque([1])
while q:
    node = q.popleft()
    if node not in visited:
        visited.add(node)
        q.extend(adj[node] - visited)
print(visited)   # {1,2,3,4,5}
```

Q91. How do you use comprehensions for image pixel processing?

Treat an image as a 2D list of pixel values. Comprehensions apply transformations (threshold, invert, scale) efficiently.

```
# Simulate 4x4 grayscale image (0-255)
image = [[100,200, 50,255],
         [ 30,180,220, 10],
         [255, 90,130, 70],
         [ 20,240, 80,160]]

# Invert pixels
inverted = [[255-p for p in row] for row in image]

# Threshold: binarise at 128
binary = [[255 if p >= 128 else 0 for p in row] for row in image]

# Flatten to 1D array
flat = [p for row in image for p in row]
avg = sum(flat) / len(flat)
print(f'Average brightness: {avg:.1f}')   # 131.9
```

Q92. How do you implement a sliding window frequency counter using dict comprehensions?

Use a dict comprehension over a sliding window (deque or slice) of the data.

```
from collections import Counter, deque

data = [1,2,3,2,1,4,3,2,1,2]
W = 4   # window size

# Frequency for each window position
```

```
windows = [
    dict(Counter(data[i:i+W]))
    for i in range(len(data) - W + 1)
]
for i, w in enumerate(windows):
    print(f'Window {i}: {w}')
# Window 0: {1:1,2:1,3:1} ...
```

Q93. How do you lazily evaluate a dict comprehension?

A dict comprehension is eager — it evaluates all key-value pairs immediately. For lazy evaluation, use a generator of tuples and pass to dict() only when needed, or use a class with __missing__.

```
# Eager dict comprehension
d = {x: x**2 for x in range(5)}   # all computed now

# Lazy: generator of pairs, dict() only when consumed
gen = ((x, x**2) for x in range(5))   # lazy
d_lazy = dict(gen)   # triggers evaluation

# Laziest: __missing__ on demand
class LazySquares(dict):
    def __missing__(self, key):
        self[key] = key ** 2
        return self[key]

ls = LazySquares()
print(ls[10])   # 100  — computed on first access
print(ls[10])   # 100  — cached thereafter
```

Q94. How do comprehensions handle None and falsy values?

None and other falsy values (0, '', [], False) pass through comprehensions normally unless explicitly filtered. Use 'if x is not None' or 'if x' to exclude them.

```
data = [1, None, 0, '', False, 2, [], 3]

# Keep all (including falsy)
all_vals = [x for x in data]
print(all_vals)   # [1,None,0,'',False,2,[],3]

# Remove None only
no_none = [x for x in data if x is not None]
print(no_none)   # [1,0,'',False,2,[],3]

# Remove all falsy
truthy = [x for x in data if x]
print(truthy)   # [1, 2, 3]
```

Q95. How do you create a power set using set and list comprehensions?

Generate all 2^n subsets by iterating over bitmasks from 0 to 2^n - 1, including an element if its corresponding bit is set.

```
def power_set(s):
    lst = list(s)
    n = len(lst)
    return [
        {lst[i] for i in range(n) if mask & (1 << i)}
        for mask in range(1 << n)
    ]

print(power_set({1,2,3}))
# [set(),{1},{2},{1,2},{3},{1,3},{2,3},{1,2,3}]

ps = power_set({'a','b','c'})
print(len(ps))   # 8 = 2^3
```

Q96. How do you use comprehensions to implement a simple cache/memoize?

Pre-compute results for a known range of inputs using a dict comprehension, giving O(1) lookup for any input in that range.

```
import math

# Pre-compute log lookup table
LOG_TABLE = {x: round(math.log(x), 6) for x in range(1, 1001)}

def fast_log(n):
    return LOG_TABLE.get(n, math.log(n))   # fallback for out-of-range

print(fast_log(100))   # 4.60517
print(fast_log(500))   # 6.214608
```

Q97. How do you chain comprehensions for multi-step transformations?

Assign intermediate comprehensions to named variables for clarity, then pass one into the next. Each step is explicit and testable.

```
import json

raw_json = '[{"name":" Alice ","score":"90"},'
raw_json += '{"name":"BOB","score":"invalid"},'
raw_json += '{"name":" Carol ","score":"85"}]'

records = json.loads(raw_json)

# Step 1: normalise names
step1 = [{**r, 'name': r['name'].strip().title()} for r in records]

# Step 2: parse scores (filter invalid)
def try_int(v):
    try: return int(v)
    except: return None
```

```
step2 = [{**r, 'score': try_int(r['score'])} for r in step1]

# Step 3: keep valid records
clean = [r for r in step2 if r['score'] is not None]
print(clean)
# [{'name':'Alice','score':90},{'name':'Carol','score':85}]
```

Q98. How do you use comprehensions to implement a Sieve of Eratosthenes?

The classic sieve requires marking composites iteratively, which does not map cleanly to a pure comprehension. A readable hybrid uses a set comprehension for composite removal.

```
def sieve(n):
    composites = {
        multiple
        for p in range(2, int(n**0.5) + 1)
        for multiple in range(p*p, n+1, p)
    }
    return [x for x in range(2, n+1) if x not in composites]

print(sieve(50))
# [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47]
```

Q99. What are the readability best practices for comprehensions?

Keep comprehensions simple and purposeful: max 2 for-clauses, short expressions, meaningful variable names. Extract complex logic to named functions. Prefer generator expressions for large data or single-pass use.

```
# TOO COMPLEX — hard to read
# r = [f(x) for a in b for x in a if g(x) and h(x) or k(x)]

# BETTER: extract helpers and split stages
def is_valid(x): return x > 0 and x % 2 == 0
def transform(x): return x ** 2 + 1

data = [[1,-2,3],[4,-5,6]]
# Flatten
flat  = [x for row in data for x in row]
# Filter
valid = [x for x in flat if is_valid(x)]
# Transform
result = [transform(x) for x in valid]
print(result)   # [5, 17, 37]
```

Q100. Putting it all together — a production-grade data processing pipeline using all three comprehension types.

Combine list, dict, and set comprehensions with generators and walrus operator to build a clean, memory-efficient analytics pipeline.

```
from collections import Counter
import re, statistics

# Raw log entries
logs = [
    'ERROR 2025-01-01 user=alice code=500 path=/api/data',
    'INFO  2025-01-01 user=bob   code=200 path=/api/users',
    'ERROR 2025-01-02 user=alice code=404 path=/api/items',
    'WARN  2025-01-02 user=carol code=429 path=/api/data',
    'ERROR 2025-01-03 user=bob   code=500 path=/api/data',
]

# 1. Parse: list comprehension -> list of dicts
def parse(line):
    fields = dict(re.findall(r'(\w+)=(\S+)', line))
    fields['level'] = line.split()[0]
    return fields

parsed = [parse(line) for line in logs]

# 2. Filter errors: list comprehension
errors = [r for r in parsed if r['level'] == 'ERROR']

# 3. Unique users with errors: set comprehension
error_users = {r['user'] for r in errors}
print('Users with errors:', error_users)

# 4. Error count per path: dict comprehension
paths = {r['path'] for r in errors}
error_by_path = {
    path: len([r for r in errors if r['path'] == path])
    for path in paths
}
print('Errors by path:', error_by_path)

# 5. Status codes: Counter from generator expression
code_freq = Counter(int(r['code']) for r in parsed)
print('Status codes:', dict(code_freq))

# 6. Summary dict: dict comprehension over levels
levels = {r['level'] for r in parsed}
summary = {
    lvl: len([r for r in parsed if r['level'] == lvl])
    for lvl in levels
}
print('Summary:', summary)
# {'ERROR':3,'INFO':1,'WARN':1}
```